Hosting Application Components

Host Site:

The application components will be hosted using a scalable cloud infrastructure on Amazon Web Services (AWS), enforcing a strict separation between the presentation, processing, and data storage layers.

- **Presentation Layer (Front End):** The user interfaces, including the Simulation Interface and Manual Control Dashboard, will be built using React and styled with Tailwind CSS to efficiently manage real-time visual updates. This front-end application will be hosted on an **AWS S3 Bucket** configured for static website hosting, utilizing Amazon CloudFront for secure and fast content delivery.
- **Business Logic Layer (Back End):** The Python Core (encompassing the Computer Vision Module, Traffic Signal Controller, Violation Detection Manager, and Fail-Safe Monitor) and a Flask application will be deployed on an **AWS EC2 (Elastic Compute Cloud) Instance**. To support the OpenCV image processing requirements, the instance will be provisioned with a minimum of 4 GB to 8 GB of RAM.
- **Data Access & Database Layer:** The database components will be hosted on an **AWS RDS (Relational Database Service) Instance**. This will run a managed MySQL 5.7+ database environment, isolating the physical raw data (Vehicle Counts, Violation Records, System Logs, and Auth Users) from the application code.

Deployment Strategy:

- **Infrastructure Provisioning:** Initialize an Ubuntu-based AWS EC2 instance for the backend environment, an AWS RDS MySQL 5.7+ instance for the database, and an AWS S3 bucket for the React front end. Configure AWS Security Groups to allow internal communication between the EC2 instance and the RDS database, while restricting direct external access to the database.

- **Server Configuration:** Install the Python runtime environment, OpenCV dependencies, and the MySQL-Python driver on the EC2 instance.
- **API & Communication Configuration:** Deploy the Flask application on the EC2 instance to act as a bridge, exposing internal RESTful APIs. The React front end will be configured to make HTTP requests to these Flask endpoints. When a user interacts with the Manual Control Dashboard, React will route the commands through the API to the Traffic Signal Controller.
- **Database Connection:** The Data Access Layer's Database Connector will be configured with the secure AWS RDS endpoint credentials to translate high-level requests into low-level SQL queries via the Query Manager.
- **Process Management:** Utilize a WSGI server (like Gunicorn) and a process manager (such as systemd) on the EC2 instance to ensure the Flask API and Business Logic Layer run continuously, automatically restarting upon failure to meet the 99.9% uptime target.

Security Measures:

To ensure system integrity and prevent unauthorized access, the following security mechanisms will be implemented:

- **Network Security:** All database connections between the Python-based EC2 backend and the RDS MySQL database will be fully encrypted.
- **Authentication & Access Control:** The React dashboard will require strict username and password authentication verified through the Flask backend. Administrative privileges will be strictly required for any system configuration changes.
- **Session Management:** To prevent unauthorized terminal access, the system will enforce a 15-minute session timeout for inactivity.

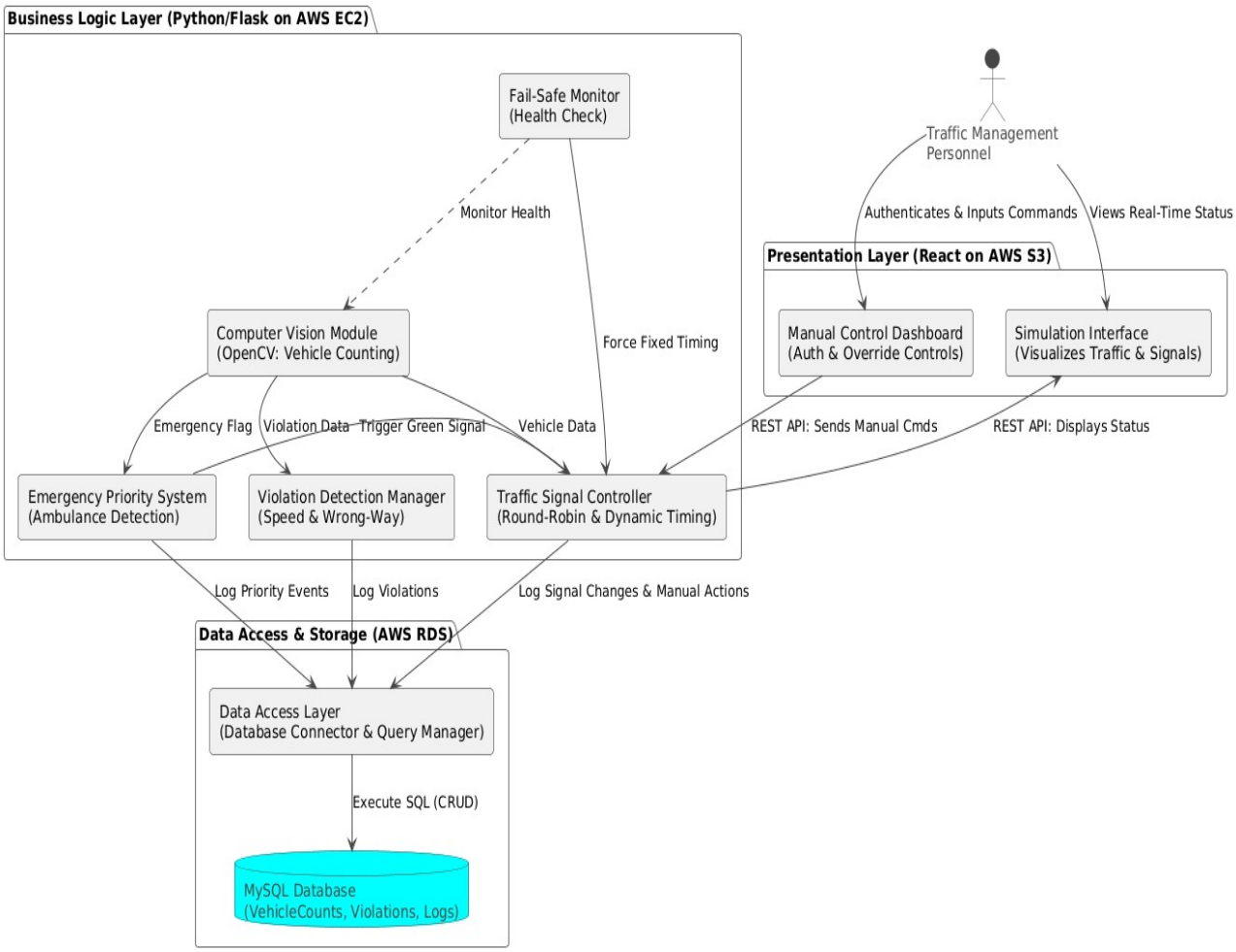
Additionally, a failed login limit of 3 attempts will trigger a temporary account lockout.

- **Audit Logging:** Comprehensive audit logs will be maintained for all critical operations, manual control actions, and data access, with every manual action securely logged alongside the user ID and timestamp.

End-User Access & System Interaction

How End Users Access the Services**:**

The system is designed to be accessed securely by authorized traffic management personnel. End users will interact with the application components through the following workflow:

- **Web Browser Access:** Users will access the React-based Presentation Layer (hosted on AWS S3/CloudFront) via a standard web browser. This layer provides the Manual Control Dashboard and the Simulation Interface.
- **Authentication:** Before interacting with the system, users must authenticate using a username and password. Based on their permission levels (view-only, partial control, or full control), the dashboard will render the appropriate interactive elements.
- **Interaction:** Once authenticated, users can view real-time traffic representations, system status dashboards, and camera views through the Simulation Interface. Through the Manual Control Dashboard, they can issue commands to override automatic operations and control individual signals (red/yellow/green).
- **API Communication:** The React front end captures these user inputs and sends them as HTTPS requests to the RESTful APIs exposed by the Flask backend running on the AWS EC2 instance. The API routes these requests to the Traffic Signal Controller for immediate execution within the 0.5-second performance requirement.

Pictorial Representation: